



Universidade Federal de Goiás
Instituto de Matemática e Estatística
Bacharelado em Matemática Aplicada e Computacional

Componente Curricular: INF0286 - ALGORITMOS E ESTRUTURAS DE DADOS 1
Turma A: 2025/2
Professora: Dra. Renata Dutra Braga

Lista 06 (30/09/25)

1) Implemente uma pilha de float com vetor fixo (MAX). Funções: init, empty, full, push, pop. No main, empilhe 5 temperaturas (ex.: 36.5, 37.2, 38.0, 36.8, 37.9) e depois desempilhe mostrando a ordem LIFO.

Sugestões de técnicas (C) e estrutura.

- `typedef struct { float v[MAX]; int topo; } Stack; | checar under/overflow | <stdio.h> <stdbool.h>`

2) Reaproveite a pilha do exercício 1 e acrescente `size(Stack*)` que retorna a quantidade de elementos. No main, empilhe 3 valores e mostre o tamanho antes e depois de um pop.

Sugestões:

- `return s->topo + 1;`

3) Leia 3 comandos (strings curtas) e empilhe. Em seguida, faça “desfazer” imprimindo os comandos na ordem inversa (LIFO).

Sugestões:

- Pilha de `char[LEN]`.

4) Implemente fila circular com `enqueue/dequeue`. Enfileire 4 números, desenfileire 2, enfileire mais 3 e, por fim, esvazie a fila imprimindo a ordem.

Sugestões:

- campos `in`, `out`, `cnt`.

5) Crie uma fila circular de Ticket `{int id; char assunto[40];}`. Enfileire 3 tickets e atenda todos em ordem FIFO, imprimindo `id` e `assunto`.

Sugestões:

- Fila de structs; usar `snprintf`.

6) Coloque os números 1, 2, 3 em uma fila. Execute 8 passos: a cada passo, retire o primeiro, imprima, e reenfileire o mesmo número ao final.

Sugestões:

- dequeue seguido de enqueue.

7) Implemente uma lista sequencial de int com size atual. Funções: insert_at(L, idx, x) (insere deslocando à direita se houver espaço) e print. No main, comece com {10,20,30} e insira 99 na posição 1, imprimindo o resultado.

Sugestões:

- Vetor a[MAX] + size.

8) Implemente insert_back e print para lista de int. Insira 4, depois 6, depois 8 (no fim) e imprima.

Sugestões:

- Percorrer até next == NULL.

9) Com a lista do exercício 10, implemente min_value(Node* h, int* out) que coloca em *out o menor valor da lista (retorna 0 se vazia).

Sugestões:

- Percorrer comparando.

10) Implemente remove_all(Node** h, int x) que remove todas as ocorrências de x. Demonstre com a lista 1 -> 2 -> 2 -> 3 -> 2 -> 4, removendo 2.

Sugestões:

- Tratar cabeça repetida com laço; depois varrer com prev/curr.